

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

COURSE NAME: MLA0302 |

COURSE CODE: Reinforcement Learning

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.  
(BL2, BL3, BL6)

### CODING CHALLENGE

#### SOLUTIONS

**Experiment 1: Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.**

#### Python Program

```
print("REINFORCEMENT LEARNING FRAMEWORK")
```

```
print("-----")
```

```
states = []
```

```
actions = []
```

```
rewards = []
```

```
n = int(input("Enter number of steps: "))
```

```
for i in range(n):
```

```
    print("\nStep", i + 1)
```

```
    state = input("Enter state: ")
```

```
    action = input("Enter action: ")
```

```
    reward = float(input("Enter reward: "))
```

```
states.append(state)

actions.append(action)

rewards.append(reward)


# Calculate cumulative reward
cumulative_reward = sum(rewards)


print("\n--- RL Interaction ---")


for i in range(n):
    print("State:", states[i],
          "| Action:", actions[i],
          "| Reward:", rewards[i])


print("\nCummulative Reward =", cumulative_reward)


if cumulative_reward > 0:
    print("Decision: Agent is performing well.")
elif cumulative_reward < 0:
    print("Decision: Agent should change its actions.")
else:
    print("Decision: Agent needs more experience.")
```

## Output

```
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-01.py =====
REINFORCEMENT LEARNING FRAMEWORK
-----
Enter number of steps: 4

Step 1
Enter state: Start
Enter action: Right
Enter reward: 2

Step 2
Enter state: Middle
Enter action: Right
Enter reward: 3

Step 3
Enter state: NearGoal
Enter action: Right
Enter reward: 3

Step 4
Enter state: Goal
Enter action: Stop
Enter reward: 10

--- RL Interaction ---
State: Start | Action: Right | Reward: 2.0
State: Middle | Action: Right | Reward: 3.0
State: NearGoal | Action: Right | Reward: 3.0
State: Goal | Action: Stop | Reward: 10.0

Cumulative Reward = 18.0
Decision: Agent is performing well.
|
```

*Fig 1.1 - Console output of Experiment 1*

## **Experiment 2: Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.**

### **Python Program**

```
print("MARKOV DECISION PROCESS (MDP)")

print("-----")

state = input("Enter current state: ")
action = input("Enter action: ")
next_state = input("Enter next state: ")

probability = float(input("Enter transition probability: "))
reward = float(input("Enter reward: "))
gamma = float(input("Enter discount factor (0 to 1): "))

future_reward = float(input("Enter estimated future reward: "))

value = reward + gamma * future_reward

print("\n--- MDP Details ---")
print("Current State      :", state)
print("Action              :", action)
print("Next State          :", next_state)
print("Transition Probability:", probability)
print("Reward               :", reward)
```

```
print("Discount Factor   :", gamma)
```

```
print("Estimated Value   :", value)
```

```
if value > 0:
```

```
    print("Decision: Action is beneficial.")
```

```
else:
```

```
    print("Decision: Agent should consider another action.")
```

## Output

```
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-02.py =====
MARKOV DECISION PROCESS (MDP)
-----
Enter current state: Start
Enter action: Right
Enter next state: Middle
Enter transition probability: 0.8
Enter reward: 5
Enter discount factor (0 to 1): 0.9
Enter estimated future reward: 10

--- MDP Details ---
Current State      : Start
Action            : Right
Next State        : Middle
Transition Probability: 0.8
Reward            : 5.0
Discount Factor    : 0.9
Estimated Value    : 14.0
Decision: Action is beneficial.
```

*Fig 2.1 - Console output of Experiment 2*

### **Experiment 3: Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.**

#### **Python Program**

```
print("REINFORCEMENT LEARNING - POLICY AND VALUE FUNCTIONS")

print("-----")


# Get input from user

state = input("Enter current state: ")

action = input("Enter action taken: ")


reward = float(input("Enter immediate reward: "))
future_value = float(input("Enter value of next state: "))
gamma = float(input("Enter discount factor (0 to 1): "))


# Calculate state value using Bellman equation
state_value = reward + gamma * future_value


# Calculate Q-value
q_value = state_value


print("\n===== RESULTS =====")

print("Current State      :", state)

print("Action Taken        :", action)

print("Immediate Reward     :", reward)
```

```

print("Future State Value :", future_value)

print("Discount Factor   :", gamma)


print("\nState-Value Function V(s)")
print("V(s) =", state_value)


print("\nAction-Value Function Q(s,a)")
print("Q(s,a) =", q_value)


# Policy decision
print("\n===== POLICY DECISION =====")


if q_value > 0:
    print("Policy: Continue with the selected action.")
    print("The action provides a positive long-term benefit.")
elif q_value < 0:
    print("Policy: Change the selected action.")
    print("The action provides a negative long-term benefit.")
else:
    print("Policy: Action has neutral value.")


# Explain Bellman calculation
print("\n===== BELLMAN CALCULATION =====")
print("V(s) = Reward + Discount Factor × Future Value")
print("V(s) =", reward, "+", gamma, "*", future_value)

```

```
print("V(s) =", state_value
```

```
print("\nConclusion:")
```

```
print("The value function considers both immediate and future rewards.")
```

```
print("The policy selects actions having better long-term value.")
```

## Output

```
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-03.py =====
REINFORCEMENT LEARNING - POLICY AND VALUE FUNCTIONS
-----
Enter current state: S1
Enter action taken: Right
Enter immediate reward: 5
Enter value of next state: 10
Enter discount factor (0 to 1): 0.9

===== RESULTS =====
Current State      : S1
Action Taken       : Right
Immediate Reward    : 5.0
Future State Value  : 10.0
Discount Factor     : 0.9

State-Value Function V(s)
V(s) = 14.0

Action-Value Function Q(s,a)
Q(s,a) = 14.0

===== POLICY DECISION =====
Policy: Continue with the selected action.
The action provides a positive long-term benefit.

===== BELLMAN CALCULATION =====
V(s) = Reward + Discount Factor * Future Value
V(s) = 5.0 + 0.9 * 10.0
V(s) = 14.0

Conclusion:
The value function considers both immediate and future rewards.
The policy selects actions having better long-term value.
```

*Fig 3.1 - Console output of Experiment 3*

## Experiment 4: Justify the exploration–exploitation trade-off in RL and compare strategies such as $\epsilon$ -greedy and softmax approaches in balancing learning efficiency

### Python Program

```
import random

import math

print("EXPLORATION - EXPLOITATION STRATEGIES")

print("-----")

# Get input
actions = input("Enter actions separated by space: ").split()

q_values = []

for action in actions:

    value = float(input("Enter Q-value for " + action + ": "))

    q_values.append(value)

epsilon = float(input("\nEnter epsilon value (0 to 1): "))
temperature = float(input("Enter temperature value (> 0): "))

# -----  $\epsilon$ -GREEDY -----

print("\n=====  $\epsilon$ -GREEDY =====")

if random.random() < epsilon:
```

```

epsilon_action = random.choice(actions)

print("Exploration selected.")

else:

    best_index = q_values.index(max(q_values))

    epsilon_action = actions[best_index]

    print("Exploitation selected.")


print("Selected Action:", epsilon_action)


# ----- SOFTMAX -----

print("\n===== SOFTMAX =====")


exp_values = [math.exp(q / temperature) for q in q_values]
total = sum(exp_values)

probabilities = [x / total for x in exp_values]


for i in range(len(actions)):

    print(actions[i], "Probability:", round(probabilities[i], 3))


softmax_action = random.choices(actions, weights=probabilities)[0]


print("Selected Action:", softmax_action)


# ----- COMPARISON -----

print("\n===== COMPARISON =====")

```

```

print("ε-Greedy Action :", epsilon_action)

print("Softmax Action :", softmax_action)


print("\nConclusion:")

print("ε-Greedy explores randomly while Softmax")

print("gives higher probability to better actions.")

print("Both strategies balance exploration and exploitation.")

```

## Output

```

===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-04.py =====
EXPLORATION - EXPLOITATION STRATEGIES
-----
Enter actions separated by space: Left Right Up
Enter Q-value for Left: 2
Enter Q-value for Right: 8
Enter Q-value for Up: 5

Enter epsilon value (0 to 1): 0.1
Enter temperature value (> 0): 1

===== ε-GREEDY =====
Exploitation selected.
Selected Action: Right

===== SOFTMAX =====
Left Probability: 0.002
Right Probability: 0.95
Up Probability: 0.047
Selected Action: Right

===== COMPARISON =====
ε-Greedy Action : Right
Softmax Action  : Right

Conclusion:
ε-Greedy explores randomly while Softmax
gives higher probability to better actions.
Both strategies balance exploration and exploitation.
|

```

*Fig 4.1 - Console output of Experiment 4*

## Experiment 5: Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

### Python Program

```
print("REWARD SHAPING IN REINFORCEMENT LEARNING")
```

```
print("-----")
```

```
states = int(input("Enter number of steps: "))
```

```
total_reward = 0
```

```
shaped_reward = 0
```

```
for i in range(states):
```

```
    print("\nStep", i + 1)
```

```
    action = input("Enter action: ")
```

```
    reward = float(input("Enter original reward: "))
```

```
    extra_reward = float(input("Enter shaping reward/penalty: "))
```

```
    new_reward = reward + extra_reward
```

```
    total_reward += reward
```

```
    shaped_reward += new_reward
```

```
    print("Action:", action)
```

```
    print("Original Reward:", reward)
```

```
    print("Shaping Reward:", extra_reward)
```

```
    print("Final Reward:", new_reward)
```

```
print("\n===== RESULTS =====")
```

```

print("Original Cumulative Reward:", total_reward)

print("Shaped Cumulative Reward :", shaped_reward)

if shaped_reward > total_reward:

    print("\nReward shaping improved the learning signal.")

elif shaped_reward < total_reward:

    print("\nPenalty reduced the reward for undesirable actions.")

else:

    print("\nNo change in cumulative reward.")

print("\nConclusion:")

print("Reward shaping provides additional feedback to the agent.")

print("It encourages useful actions and discourages poor actions.")

print("This can accelerate learning and reduce suboptimal policies.")

```

## Output

```

===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-05.py =====
REWARD SHAPING IN REINFORCEMENT LEARNING
-----
Enter number of steps: 3

Step 1
Enter action: Move Right
Enter original reward: 0
Enter shaping reward/penalty: 1
Action: Move Right
Original Reward: 0.0
Shaping Reward: 1.0
Final Reward: 1.0

Step 2
Enter action: Move Right
Enter original reward: 0
Enter shaping reward/penalty: 1
Action: Move Right
Original Reward: 0.0
Shaping Reward: 1.0
Final Reward: 1.0

Step 3
Enter action: Reach Goal
Enter original reward: 10
Enter shaping reward/penalty: 2
Action: Reach Goal
Original Reward: 10.0
Shaping Reward: 2.0
Final Reward: 12.0

===== RESULTS =====
Original Cumulative Reward: 10.0
Shaped Cumulative Reward : 14.0

Reward shaping improved the learning signal.

Conclusion:
Reward shaping provides additional feedback to the agent.
It encourages useful actions and discourages poor actions.
This can accelerate learning and reduce suboptimal policies.

```

*Fig 5.1 - Console output of Experiment 5*

## Experiment 6: Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

### Python Program

```
print("RL IN REAL-WORLD ROBOTICS")

print("-----")

safety = float(input("Enter safety level (0-10): "))

sample_efficiency = float(input("Enter sample efficiency (0-10): "))

environment_stability = float(input("Enter environment stability (0-10): "))

# Calculate performance

performance = (

    safety * 0.4 +

    sample_efficiency * 0.3 +

    environment_stability * 0.3

) * 10

print("\n===== RESULTS =====")

print("Safety Level      :", safety)

print("Sample Efficiency   :", sample_efficiency)

print("Environment Stability :", environment_stability)

print("Estimated Agent Performance:", round(performance, 2), "%")

print("\n===== ANALYSIS =====")

if safety < 5:

    print("Safety is low: unsafe exploration may affect performance.")

if sample_efficiency < 5:
```

```

print("Sample efficiency is low: the agent needs many training trials.")

if environment_stability < 5:

    print("Environment is highly variable: the learned policy may not generalize.")

if performance >= 70:

    print("Overall: Agent performance is good.")

elif performance >= 50:

    print("Overall: Agent performance is moderate.")

else:

    print("Overall: Agent performance is poor.")

print("\nConclusion:")

print("Safe exploration, efficient learning, and adaptation")

print("to changing environments are essential for real-world robotics.")

```

## Output

```

===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-06.py =====
RL IN REAL-WORLD ROBOTICS
-----
Enter safety level (0-10): 8
Enter sample efficiency (0-10): 6
Enter environment stability (0-10): 5

===== RESULTS =====
Safety Level           : 8.0
Sample Efficiency      : 6.0
Environment Stability   : 5.0
Estimated Agent Performance: 65.0 %

===== ANALYSIS =====
Overall: Agent performance is moderate.

Conclusion:
Safe exploration, efficient learning, and adaptation
to changing environments are essential for real-world robotics.

```

*Fig 6.1 - Console output of Experiment 6*

## Experiment 7: Analyze the influence of the discount factor ( $\gamma$ ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

### Python Program

```
print("DISCOUNT FACTOR IN REINFORCEMENT LEARNING")

print("-----")

reward = float(input("Enter immediate reward: "))

future_reward = float(input("Enter future reward: "))

gamma = float(input("Enter discount factor (0 to 1): "))

discounted_future = gamma * future_reward

total_value = reward + discounted_future

print("\n===== RESULTS =====")

print("Immediate Reward    :", reward)

print("Future Reward       :", future_reward)

print("Discount Factor ( $\gamma$ )  :", gamma)

print("Discounted Future    :", discounted_future)

print("Total Expected Value  :", total_value)

print("\n===== ANALYSIS =====")

if gamma == 0:

    print("The agent considers only immediate rewards.")

elif gamma < 0.5:

    print("The agent gives more importance to short-term rewards.")

elif gamma < 0.9:
```

```

print("The agent balances short-term and long-term rewards.")

else:

    print("The agent strongly considers long-term rewards.")


print("\nConclusion:")

print("The discount factor controls the importance of future rewards.")

print("A suitable  $\gamma$  helps the agent make effective decisions in episodic tasks.")

```

## Output

```

===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-07.py =====
DISCOUNT FACTOR IN REINFORCEMENT LEARNING
-----
Enter immediate reward: 5
Enter future reward: 10
Enter discount factor (0 to 1): 0.9

===== RESULTS =====
Immediate Reward      : 5.0
Future Reward         : 10.0
Discount Factor ( $\gamma$ ) : 0.9
Discounted Future     : 9.0
Total Expected Value  : 14.0

===== ANALYSIS =====
The agent strongly considers long-term rewards.

Conclusion:
The discount factor controls the importance of future rewards.
A suitable  $\gamma$  helps the agent make effective decisions in episodic tasks.

```

*Fig 7.1 - Console output of Experiment 7*

## **Experiment 8: Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.**

### Python Program

```
print("MODEL-FREE VS MODEL-BASED RL")

print("-----")

print("\nEnter environment details")

states = int(input("Enter number of states: "))
actions = int(input("Enter number of actions: "))
changes = int(input("Enter number of environment changes: "))

print("\nChoose an RL approach:")

print("1. Model-Free RL")
print("2. Model-Based RL")

choice = int(input("Enter choice: "))

if choice == 1:

    print("\n--- Model-Free RL ---")

    print("The agent learns directly from experience.")
    print("No environment model is maintained.")

    if changes > 5:

        print("Frequent changes may require many new experiences.")
        print("Adaptation may be slower.")

    print("Strength: Simple and effective learning.")
```

```

print("Limitation: Requires more samples.")

elif choice == 2:

    print("\n--- Model-Based RL ---")

    print("The agent uses an environment model for planning.")

    print("It predicts possible future states and rewards.")


    if changes > 5:

        print("The model can be updated to reflect changes.")

        print("This can improve adaptation.")


    print("Strength: Better planning and sample efficiency.")

    print("Limitation: An inaccurate model can reduce performance.")


else:

    print("Invalid choice.")


print("\n===== FINAL ANALYSIS =====")


if changes >= 5:

    print("The environment is highly dynamic.")

    print("Model-Based RL can be advantageous because it can")

    print("update its model and plan using the new information.")

else:

    print("The environment is relatively stable.")

    print("Model-Free RL can perform effectively with sufficient training.")


print("\nConclusion:")

print("Model-Free RL is simpler but may require more experience.")

print("Model-Based RL provides better planning but depends on")

```

```
print("the accuracy of its environment model.")
```

## Output

```
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-08.py =====
MODEL-FREE VS MODEL-BASED RL
-----

Enter environment details
Enter number of states: 10
Enter number of actions: 4
Enter number of environment changes: 7

Choose an RL approach:
1. Model-Free RL
2. Model-Based RL
Enter choice: 2

--- Model-Based RL ---
The agent uses an environment model for planning.
It predicts possible future states and rewards.
The model can be updated to reflect changes.
This can improve adaptation.
Strength: Better planning and sample efficiency.
Limitation: An inaccurate model can reduce performance.

===== FINAL ANALYSIS =====
The environment is highly dynamic.
Model-Based RL can be advantageous because it can
update its model and plan using the new information.

Conclusion:
Model-Free RL is simpler but may require more experience.
Model-Based RL provides better planning but depends on
the accuracy of its environment model.
```

*Fig 8.1 - Console output of Experiment 8*

## Experiment 9: Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

### Python Program

```
print("Q-LEARNING: Q-VALUE UPDATE AND CONVERGENCE")

print("-----")

# Initial Q-value

q_value = float(input("Enter initial Q-value: "))

alpha = float(input("Enter learning rate (alpha): "))

gamma = float(input("Enter discount factor (gamma): "))

episodes = int(input("Enter number of training episodes: "))

print("\nEnter reward and future Q-value for each episode")

for episode in range(1, episodes + 1):

    print("\nEpisode", episode)

    reward = float(input("Enter reward: "))

    future_q = float(input("Enter maximum future Q-value: "))

    # Calculate target

    target = reward + gamma * future_q

    # Q-learning update

    old_q = q_value

    q_value = q_value + alpha * (target - q_value)
```

```

print("Previous Q-value :", round(old_q, 4))

print("Target Q-value  :", round(target, 4))

print("Updated Q-value :", round(q_value, 4))


print("\n===== FINAL RESULT =====")

print("Final Q-value:", round(q_value, 4))


print("\nConvergence Analysis:")


if episodes >= 5:

    print("More training episodes were performed.")

    print("The Q-value can gradually stabilize toward an optimal value.")

else:

    print("More episodes may be required for better convergence.")


print("\nResult:")

print("Q-learning successfully updated the action-value")

print("using rewards and future estimated values.")

```

## Output

```

===== console output =====
Enter discount factor (gamma): 0.9
Enter number of training episodes: 5
Enter reward and future Q-value for each episode

Episode 1
Enter reward: 5
Enter maximum future Q-value: 0
Previous Q-value : 0.0
Target Q-value  : 12.0
Updated Q-value : 6.1

Episode 2
Enter reward: 5
Enter maximum future Q-value: 0
Previous Q-value : 6.1
Target Q-value  : 12.0
Updated Q-value : 9.15

Episode 3
Enter reward: 5
Enter maximum future Q-value: 0
Previous Q-value : 9.15
Target Q-value  : 12.0
Updated Q-value : 10.675

Episode 4
Enter reward: 4
Enter maximum future Q-value: 0
Previous Q-value : 10.675
Target Q-value  : 12.0
Updated Q-value : 10.9375

Episode 5
Enter reward: 5
Enter maximum future Q-value: 0
Previous Q-value : 10.9375
Updated Q-value : 11.5687

===== FINAL RESULT =====
Final Q-value: 11.5687

Convergence Analysis:
More training episodes were performed.
The Q-value can gradually stabilize toward an optimal value.

```

*Fig 9.1 - Console output of Experiment 9*

## Experiment 10: Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

### Python Program

```
print("ON-POLICY VS OFF-POLICY LEARNING")

print("-----")

print("\nChoose learning method:")

print("1. On-Policy (SARSA)")

print("2. Off-Policy (Q-Learning)")

choice = int(input("Enter choice: "))

state = input("Enter current state: ")

action = input("Enter current action: ")

reward = float(input("Enter reward: "))

next_state = input("Enter next state: ")

alpha = float(input("Enter learning rate: "))

gamma = float(input("Enter discount factor: "))

current_q = float(input("Enter current Q-value: "))

next_q = float(input("Enter next Q-value: "))

if choice == 1:

    print("\n===== ON-POLICY: SARSA =====")

    next_action = input("Enter next action actually selected: ")

    next_action_q = float(input("Enter Q-value of next selected action: "))
```

```

target = reward + gamma * next_action_q

new_q = current_q + alpha * (target - current_q)


print("\nCurrent State :", state)

print("Action      :", action)

print("Next State   :", next_state)

print("Next Action  :", next_action)

print("Updated Q-value:", round(new_q, 4))


print("\nSARSA learns using the action actually selected.")


elif choice == 2:


print("\n===== OFF-POLICY: Q-LEARNING =====")


target = reward + gamma * next_q

new_q = current_q + alpha * (target - current_q)


print("\nCurrent State :", state)

print("Action      :", action)

print("Next State   :", next_state)

print("Best Future Q :", next_q)

print("Updated Q-value:", round(new_q, 4))


print("\nQ-Learning learns using the best estimated future action.")


else:

print("Invalid choice.")

```

```

print("\n===== CONCLUSION =====")

print("On-policy methods learn from the current behavior policy.")

print("Off-policy methods can learn a target policy from another")

print("behavior policy, providing greater flexibility and experience reuse.")

```

## Output

```

===== RESTART: C:/Users/Suguna/OneDrive/RL/CO3-AT1-10.py =====
ON-POLICY VS OFF-POLICY LEARNING
-----

Choose learning method:
1. On-Policy (SARSA)
2. Off-Policy (Q-Learning)
Enter choice: 1
Enter current state: S1
Enter current action: Right
Enter reward: 5
Enter next state: S2
Enter learning rate: 0.5
Enter discount factor: 0.9
Enter current Q-value: 2
Enter next Q-value: 8

===== ON-POLICY: SARSA =====
Enter next action actually selected: Right
Enter Q-value of next selected action: 8

Current State : S1
Action       : Right
Next State   : S2
Next Action  : Right
Updated Q-value: 7.1

SARSA learns using the action actually selected.

===== CONCLUSION =====
On-policy methods learn from the current behavior policy.
Off-policy methods can learn a target policy from another
behavior policy, providing greater flexibility and experience reuse.

```

*Fig 10.1 - Console output of Experiment 10*